



► L'implémentation du nouveau **type composé** nommé **RECORD** a permis de **déclarer des variables de type enregistrement**

```
TYPE Nom_Type IS RECORD (  
  champ { typechamp | table.column%TYPE} [NOT NULL],  
  champ { typechamp | table.column%TYPE} [NOT NULL], ....)
```

```
Nom_Variable  Nom_Type ;
```

DECLARE

-- Déclaration d'un type d'enregistrement personnalisé

TYPE type_client **IS RECORD** {

nom VARCHAR2(50),

age NUMBER,

adresse VARCHAR2(100)

};

-- Déclaration d'une variable de type d'enregistrement

info_client type_client;

BEGIN

...

END;

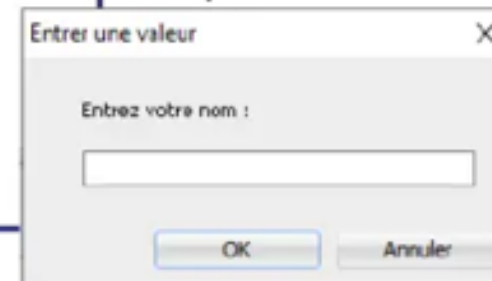


► La procédure **ACCEPT** est utilisée pour lire des valeurs saisies par l'utilisateur pendant l'exécution du programme PL/SQL

ACCEPT variable_name **PROMPT** 'texte'

Exemple 1 :

```
ACCEPT nom PROMPT 'Entrez votre nom : ' ;  
DECLARE  
  v_nom VARCHAR2(50);  
BEGIN  
  v_nom := '&nom';  
  DBMS_OUTPUT.PUT_LINE('Bonjour, ' || v_nom);  
END;  
/
```





- L'affectation d'une expression se fait par l'opérateur :=
- Outre les conversions explicites par les fonctions prédéfinies (TO_CHAR, TO_NUMBER, TO_DATE, ...), PL/SQL peut effectuer des conversions implicites lors des affectations ou évaluations d'expressions.

- ♣ Affectation d'un nombre dans une chaîne.

Ex. message `VARCHAR2(20) := 1200;`

- ♣ Affectation d'une chaîne contenant un nombre.

Ex. salaire `number(7,3) := '1200.500';`

- ♣ Conversion implicite entre DATE et CHAR.

Ex. aujourd'hui `VARCHAR2(9) := SYSDATE;`
`Vdate DATE := '16-Sep-2021';`

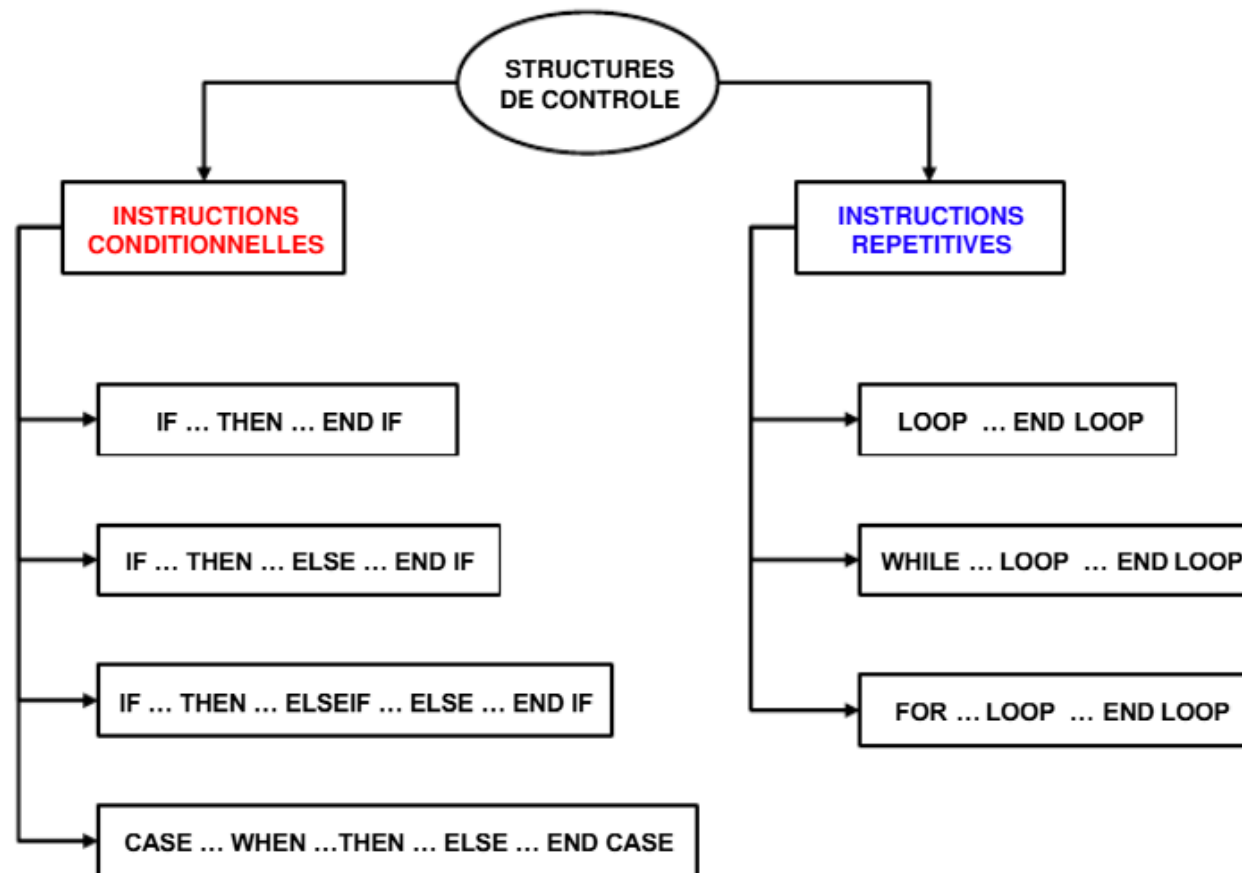


► L'évaluation des expressions repose sur les règles de priorité des opérateurs présentés de la plus haute vers la plus faible priorité :

Opérateurs	Opération
** , NOT	Puissance, négation logique
+ , -	Identité, inversion de signe
* , /	Multiplication, division
+ , - , 	Addition, soustraction, concaténation
< , <= , > , >= , = , !=	Opérateurs de comparaison
IS NULL , LIKE , BETWEEN , IN	Test de nullité, comparaison de chaîne, intervalle de valeur, appartenance
AND	ET logique
OR	OU logique



► Les principales structures de contrôle de PL/SQL sont:





- Utilisation de **ELSE** pour les structures imbriquées

```
IF condition1_PLSql
  THEN Traitement1
  [ELSE]
    [IF condition2_PLSql
      THEN Traitement2
      [ELSE TraitementN]
    END IF; ]
END IF;
```

- Utilisation de **ELSIF** pour les structures imbriquées

```
IF condition_PLSql
  THEN commandes
  [ELSE commandes]
  [ELSIF condition_PLSql
    THEN commandes
    [ELSE commandes]]
END IF;
```

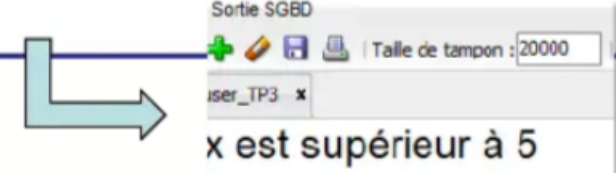


Les traitements conditionnels : L'alternative

► Exemple

```
DECLARE
    m INTEGER := 12;
BEGIN
    IF (m=1) THEN
        dbms_output.put_line('Janvier');
    ELSIF (m=2) THEN
        dbms_output.put_line('Fevrier');
        ...
    ELSIF (m=12) THEN
        dbms_output.put_line('Décembre');
    ELSE
        dbms_output.put_line('Numéro de mois erroné');
    END IF;
END;
```

```
DECLARE
    x NUMBER := 10;
BEGIN
    IF x > 5 THEN
        DBMS_OUTPUT.PUT_LINE('x est supérieur à 5');
    ELSE
        DBMS_OUTPUT.PUT_LINE('x est inférieur ou égal à 5');
    END IF;
END;
```



The screenshot shows the "Sortie SGBD" (Database Output) window in SQL Developer. It displays the output of the PL/SQL code: "x est supérieur à 5". The window title is "user_TP3 - x". The "Taille de tampon" (Buffer size) is set to 20000.



```
[<<label>>] CASE sélecteur
    WHEN expression1 THEN Trait1 ;
    WHEN expression2 THEN Trait2 ;
    ...
    ELSE TraitN;
END CASE [label];
```

► Exemple

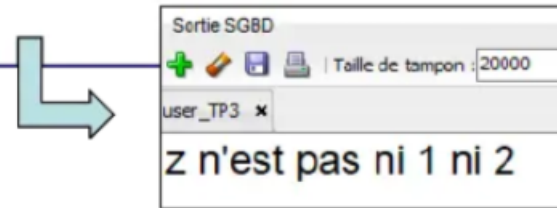
```
DECLARE
    m INTEGER := 12;
BEGIN
    <<mois>> CASE m
        WHEN 1 THEN dbms_output.put_line('Janvier');
        WHEN 2 THEN dbms_output.put_line('février');
        ...
        WHEN 12 THEN dbms_output.put_line('décembre');
        ELSE dbms_output.put_line('mois erroné');
    END CASE mois;
END;
```




```
[<<label>>] CASE sélecteur  
    WHEN expression1 THEN Trait1 ;  
    WHEN expression2 THEN Trait2 ;  
    ...  
    ELSE TraitN;  
END CASE [label];
```

► Exemple

```
DECLARE  
    z NUMBER := 3;  
BEGIN  
    CASE z  
        WHEN 1 THEN  
            DBMS_OUTPUT.PUT_LINE('z est égal à 1');  
        WHEN 2 THEN  
            DBMS_OUTPUT.PUT_LINE('z est égal à 2');  
        ELSE  
            DBMS_OUTPUT.PUT_LINE('z n'est pas ni 1 ni 2');  
    END CASE;  
END;
```





La structure Loop End Loop

```
BEGIN  
[ <<Label>> ] LOOP  
...  
Instructions  
...  
END LOOP [Label];  
END;
```

La structure While

```
[ <<Label>> ]  
WHILE condition_PLSql LOOP  
...  
Instructions  
...  
END LOOP [Label];  
END;
```

```
DECLARE  
compteur NUMBER := 1;  
BEGIN  
  WHILE compteur <= 5 LOOP  
    DBMS_OUTPUT.PUT_LINE('compteur : ' || compteur);  
    compteur := compteur + 1;  
  END LOOP;  
END;
```

A screenshot of a SQL Developer window titled "Sortie de script". It shows the output of the PL/SQL code, which is a list of numbers from 1 to 5, each preceded by "compteur :".

```
Sortie de script  
Sortie SQLDB  
user_TP3  
compteur : 1  
compteur : 2  
compteur : 3  
compteur : 4  
compteur : 5
```



```
[ <<Label>> ]  
FOR compteur IN [REVERSE] val_debut .. val_fin LOOP  
...  
Instructions  
...  
END LOOP [Label];  
END;
```

```
DECLARE  
    total NUMBER := 0;  
BEGIN  
    FOR i IN 1..5 LOOP  
        total := total + i;  
    END LOOP;  
    DBMS_OUTPUT.PUT_LINE('Total : ' || total);  
END;
```



```
--Etiquette  
<< Label >>  
.....  
--instruction GOTO  
GOTO Label ;
```

► Exemple

```
DECLARE  
    i number :=1;  
    f number:=1;  
    n number:=5;  
  
BEGIN  
    <<instruction I>>  
    f := f * i;  
    i:=i+1;  
    if (i <= n) then  
        GOTO instruction I;  
    End if;  
    dbms_output.put_line('Factoriel de '||n||' égale : ' || f);  
END;
```



Pour quitter plusieurs boucles imbriquées, il faut tout d'abord **étiqueter les boucles** puis **utiliser l'option étiquette de la commande EXIT**.

► Exemple

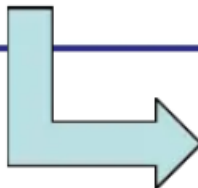
```
<<boucleI>>  
LOOP  
...  
    LOOP  
    ...  
    EXIT WHEN ...  
    EXIT boucleI WHEN ...  
    ...  
    END LOOP;  
...  
END LOOP boucleI;
```



Exercice

Ecrire un programme PL/SQL qui affiche les nombres de 1 à 10 en utilisant une boucle For. Si le nombre est pair, affichez « Pair », sinon affichez « Impair »

```
DECLARE
  v_num NUMBER;
BEGIN
  FOR v_num IN 1..10 LOOP
    IF MOD(v_num, 2) = 0 THEN
      DBMS_OUTPUT.PUT_LINE(v_num || ' : Pair');
    ELSE
      DBMS_OUTPUT.PUT_LINE(v_num || ' : Impair');
    END IF;
  END LOOP;
END;
/
```



Sortie SQL*Plus

user_TP3
1 : Impair
2 : Pair
3 : Impair
4 : Pair
5 : Impair
6 : Pair